

LEARN OOP THE HARD WAY

Authors:

Georgy ZAOUK

20250144

Bruno YOUNG DE CASTRO

20251333

Akshay KHOKAN

20224297

Supervised by :

Massinissa HAMIDI

Table of contents

I. INTRODUCTION.....	2
II. MODELING OVERVIEW.....	3
2.1 Class Diagram.....	3
2.2 Core Design Choices.....	4
III. MODEL DESCRIPTION.....	5
3.1 Base Estimator Abstraction.....	5
3.2 Supervised Models.....	6
3.2.1 Linear Regression.....	6
3.2.2 Decision Tree Classifier.....	8
3.3 Unsupervised Models.....	9
3.3.1 KMeans.....	9
3.3.2 Agglomerative Clustering.....	10
3.3.3 SpectralClustering.....	11
3.4 Datasets & Data Loading.....	12
3.5 Transformers.....	13
3.6 Model Selection.....	14
3.7 Metrics.....	15
3.8 Pipeline.....	17
IV. LIBRARY EVALUATION & PERFORMANCE.....	18
4.1 Basic tests.....	18
4.2 Performance & Comparison with Sklearn.....	19
4.2.1 Regression & Classification Comparison.....	19
4.2.2 Unsupervised Learning Comparison (Clustering).....	21
4.2.3 Computational Performance.....	25
V. CONCLUSION.....	26

Modeling and Implementing a Machine Learning Library.

Bruno YOUNG DE CASTRO, Georgy ZAOUK & Akshay KHOKAN.

I. INTRODUCTION

This project consists of designing and implementing a small machine-learning library in Python with a strong focus on object-oriented programming principles. The goal is not only to reproduce basic machine-learning functionalities, but to build a clean, modular and extensible software architecture inspired by libraries such as scikit-learn, without using external ML frameworks. Throughout the project, we will model core components such as estimators, datasets, transformers and pipelines, and carefully document our design choices, iterations and trade-offs in a technical report.

Concretely, we will implement both supervised and unsupervised learning algorithms (including decision trees, k-means, agglomerative and spectral clustering), along with dataset handlers capable of loading CSV files and standard datasets such as MNIST. We will also develop preprocessing transformers (standardization, normalization, binarization, one-hot encoding) and a pipeline system that allows chaining transformations and models in a unified workflow. Additional utilities for model selection, performance metrics, and mini-batch data loading will be provided to support training and evaluation.

Finally, the project emphasizes software engineering quality as much as algorithmic correctness. We will use advanced Python features such as dunder methods, decorators, inheritance and iterators to create a user-friendly API, implement testing through continuous integration, and compare the performance of our implementations with reference libraries such as PyTorch. The final deliverables include a fully functional library, a documented design with class diagrams and version history, and an evaluation of the strengths and limitations of the proposed architecture.

2.2 Core Design Choices

The main goal of our library is to be able to replicate some machine learning frameworks such as scikit-learn. Our modeling was based on ensuring readable implementations, and a logic object-oriented structure that will be able to accommodate both supervised and unsupervised learning paradigms.

A decision we made early on was to use a single abstract **Estimator** base class from which all learning components inherit. This abstraction enforces that all models share the same lifecycle (initialization, fitting, and application) while allowing for specific behaviors to be differentiated through inheritance. By separating estimators into **Supervised** and **Unsupervised** subclasses, the library is capable of differentiating between prediction-based models and clustering models. This avoids ambiguity in model usage and ensures that supervised and unsupervised algorithms can both be used in pipelines and evaluation workflows.

Another key design choice is the consistent separation between model configuration and learned state. Hyperparameters are defined at initialization time, while attributes created during training are stored using a trailing underscore convention (e.g., **coef_**, **labels_**, **centroids_**). We decided this by basing ourselves on existing machine learning libraries, so that we can enable clear inspection of model state to allow for clear understanding of the model and easier debugging, and also permit for estimators to be compared independently of their training.

The library also adopts a modular composition approach rather than deep inheritance hierarchies. We constructed our workflows so that each major task (dataset loading, transformers, estimators, and metrics) are done individually rather than inserting all this preprocessing and evaluation inside the model. We built this so **Transformers** inherit from the same estimator base class and will first be trained with fit and then transformed which allows the preprocessing to be easily linked to our learning algorithms. This design is reflected in the **Pipeline** class, which manages the execution of transformers followed by a final estimator while maintaining a clear separation between training and inference phases.

Data loading is deliberately kept separate from learning algorithms. All datasets expose the same **(X, y)** interface regardless of where the data is obtained from (local files, remote resources, synthetic generators, etc.). This ensures that models and pipelines remain independent from data sources and can be continuously reused without modification in all datasets.

Finally, evaluation logic is kept external to models through standalone metric functions and simple model selection utilities. This separation reinforces our main idea of separate systems and prevents evaluation metrics from affecting learning algorithms, making the system easier to extend and comprehend. New metrics, datasets, or models can be added without requiring changes to existing components, which aligns with the overall goal of building a clean and extensible object-oriented machine learning library.

III. MODEL DESCRIPTION

3.1 Base Estimator Abstraction

Estimator, Supervised, Unsupervised – upsacalay-ml/models/estimator.py

The **Estimator** base class provides a unified interface for all models, similar to scikit-learn. It enforces a common workflow: instantiate, fit, and apply a model.

```
class Estimator():

    # Abstract fit method
    def fit(self, X, y=None):
        raise NotImplementedError("fit method must be implemented by subclasses")

    # Compare estimators by class + hyperparameters. Ignore attributes ending with '_'.
    def __eq__(self, other):
        if type(self) is type(other):
            if {k_self: v_self for k_self, v_self in self.__dict__.items() if not k_self.endswith('_')} == \
                {k_other: v_other for k_other, v_other in other.__dict__.items() if not k_other.endswith('_')}:
                return True
            return False
        # or raise NotImplementedError("")

    # Call method to make the estimator callable
    def __call__(self, X, y=None):
        """
        Supervised: calls predict(X)
        Unsupervised: calls fit_predict(X)
        Transformer: calls transform(X)
        """
        if hasattr(self, "fit_predict"):
            return self.fit_predict(X, y) # unsupervised
        if hasattr(self, "predict"):
            return self.predict(X) # supervised
        if hasattr(self, "transform"):
            return self.transform(X) # transformers (optional)
        raise TypeError(f"{self.__class__.__name__} is not callable")

    # String representation (class + hyperparameters).
    def __repr__(self):
        return f"{self.__class__.__name__}" \
            f"({'', ' '.join(f'{k}={v}' for k, v in self.__dict__.items() if not k.endswith('_'))})"

    def __str__(self):
        return self.__repr__()
```

Figure 2. Estimator implementation.

All estimators must implement an abstract **fit(X, y=None)** method. The class also defines **__call__**, which automatically applies the correct operation: **predict** for supervised models, **fit_predict** for unsupervised models, and **transform** for transformers. Clear string representations (**__repr__**, **__str__**) display the class name and hyperparameters, making debugging easier. The **__eq__** method compares estimators using only their hyperparameters, ignoring learned attributes.

To distinguish learning types, the base class is extended into **Supervised** and **Unsupervised** classes. Supervised models must implement **predict(X)**, while unsupervised models provide **fit_predict(X)** and expose a **labels_** attribute after fitting. This separation keeps the API clear while remaining consistent.

3.2 Supervised Models

*LinearRegression, DecisionTreeClassifier –
[upsaclay_ml/models/linear_models.py](#), [upsaclay_ml/models/tree.py](#)*

All supervised models in the library inherit from the **Supervised** base class and therefore implement a **predict(X)** method in addition to the common **fit(X, y)** interface. These models require labeled data during training and are designed to integrate seamlessly with pipelines, preprocessing transformers, and evaluation metrics.

3.2.1 Linear Regression

The **LinearRegression** class implements a simple linear regression model trained using batch gradient descent. Its goal is to provide a clear and minimal example of a supervised learning algorithm while respecting the same API conventions as more complex models.

During training, input samples are converted into numerical vectors and the weight parameters are updated iteratively to minimize the error. The model supports an optional intercept through the **fit_intercept** parameter, allowing it to learn a bias term. Basic input validation and shape checks are performed, and learned parameters are stored in **coef_** and **intercept_**.

```
def fit(self, X, y=None):
    if X is None or y is None:
        raise ValueError("X and y cannot be None")
    n = len(X)
    if n == 0:
        raise ValueError("X cannot be empty")
    if len(y) != n:
        raise ValueError("X and y must have the same length")
    if self.lr <= 0:
        raise ValueError("lr must be > 0")
    if self.n_iter <= 0:
        raise ValueError("n_iter must be a positive integer")
    X_list = [_as_point(X[i]) for i in range(n)]
    y_list = [float(v) for v in y]
    d = len(X_list[0])
    if d == 0:
        raise ValueError("Samples must have at least 1 feature")
    for i, p in enumerate(X_list):
        if len(p) != d:
            raise ValueError(f"Inconsistent feature dimensions at index {i}: expected {d}, got {len(p)}")
    w = [0.0] * d # init params
    b = 0.0
    # gradient descent
    for _ in range(self.n_iter): # predictions and errors
        grad_w = [0.0] * d # y_hat = Xw + b
        grad_b = 0.0
        for i in range(n):
            y_hat = _dot(X_list[i], w) + (b if self.fit_intercept else 0.0)
            err = y_hat - y_list[i]
            for j in range(d):
                grad_w[j] += err * X_list[i][j]
            if self.fit_intercept:
                grad_b += err
        inv_n = 1.0 / n # average gradients (MSE derivative uses 2/n factor; we can absorb 2 into lr)
        for j in range(d):
            w[j] -= self.lr * (grad_w[j] * inv_n)
        if self.fit_intercept:
            b -= self.lr * (grad_b * inv_n)

    self.coef_ = w
    self.intercept_ = b if self.fit_intercept else 0.0
    return self
```

Figure 3. fit() method of LinearRegression implementation.

After training, predictions are computed by applying the learned linear function to new data using the **predict(X)** method. As a subclass of **Supervised**, the model integrates naturally with pipelines and evaluation metrics such as mean squared error.

3.2.2 Decision Tree Classifier

The **DecisionTreeClassifier** class implements a basic decision tree for classification with numerical features. Its purpose is to learn non-linear decision rules by splitting the feature space into simpler regions.

The tree is built step by step from the root. At each node, the algorithm tests different feature and threshold combinations and chooses the split that best reduces impurity, using either Gini index or entropy. To avoid overfitting, stopping rules such as maximum depth, minimum number of samples, and pure nodes are applied. Each node stores either a split rule or a class label if it is a leaf.

```
def _build_tree(self, X: List[List[float]], y: List[int], depth: int) -> Node:
    # stopping conditions (overfitting control mentioned in slides)
    # - pure node
    if len(set(y)) == 1:
        return Node(label=y[0])
    # - max depth
    if depth >= self.max_depth:
        return Node(label=_majority_class(y))
    # - too few samples
    if len(y) < self.min_samples_split:
        return Node(label=_majority_class(y))

    best = self._best_split(X, y)
    if best is None:
        return Node(label=_majority_class(y))

    feat, thr, left_idx, right_idx = best
    if not left_idx or not right_idx:
        return Node(label=_majority_class(y))

    left_X = [X[i] for i in left_idx]
    left_y = [y[i] for i in left_idx]
    right_X = [X[i] for i in right_idx]
    right_y = [y[i] for i in right_idx]

    node = Node(feature=feat, threshold=thr)
    node.left = self._build_tree(left_X, left_y, depth + 1)
    node.right = self._build_tree(right_X, right_y, depth + 1)
    return node
```

Figure 4. build_tree() method of DecisionTreeClassifier implementation.

For prediction, a sample moves through the tree by following the split rules until it reaches a leaf, which provides the predicted class. Learned information, such as the tree structure and observed classes, is stored in attributes ending with `_`, keeping the model consistent with the supervised estimator interface.

3.3 Unsupervised Models

*KMeans, AgglomerativeClustering, SpectralClustering –
upsaclay_ml/models/cluster.py*

All unsupervised models in the library inherit from the **Unsupervised** base class and therefore share the same interaction pattern. They expose a **fit(X)** method for training and produce cluster assignments through the **labels_** attribute, which can be retrieved directly by calling the estimator. This design ensures that different clustering algorithms can be used interchangeably inside pipelines and evaluation workflows.

3.3.1 KMeans

The **KMeans** class implements the standard k-means clustering algorithm for numerical data. Its goal is to group samples into a fixed number of clusters by repeatedly assigning them to the nearest centroid and updating these centroids.

The algorithm follows a simple assign–update loop. Centroids are first initialized by randomly choosing samples, with an optional **random_state** to control randomness. At each step, samples are assigned to the closest centroid using a chosen distance metric (Manhattan, Euclidean, or squared Euclidean). Centroids are then updated as the mean of the assigned points. The process stops when assignments no longer change or when a maximum number of iterations is reached. Empty clusters are handled by reinitializing their centroids.

After fitting, the final centroids in **centroids_**. As an **Unsupervised** model, calling the estimator directly returns the cluster labels, making it easy to use with pipelines and evaluation metrics.

```
# ---- initialize centroids: choose k distinct samples ----
init_indices = rng.sample(range(n_samples), self.n_clusters)
centroids = [X_list[idx][:] for idx in init_indices]

labels = [-1] * n_samples
old_labels = None

# ---- main loop ----
dist_fn = _get_distance(self.metric)
for _ in range(self.max_iter):
    # assignment step
    for i, p in enumerate(X_list):
        best_j = 0
        best_dist = dist_fn(p, centroids[0])
        for j in range(1, self.n_clusters):
            dist = dist_fn(p, centroids[j])
            if dist < best_dist:
                best_dist = dist
                best_j = j
        labels[i] = best_j

    # convergence check (labels unchanged)
    if old_labels is not None and labels == old_labels:
        break
    old_labels = labels[:]

    # update step
    clusters: List[List[List[float]]] = [[] for _ in range(self.n_clusters)]
    for i, lab in enumerate(labels):
        clusters[lab].append(X_list[i])

    for j in range(self.n_clusters):
        if clusters[j]:
            centroids[j] = _mean(clusters[j])
        else:
            # empty cluster: reinitialize centroid to a random data point
            centroids[j] = X_list[rng.randrange(n_samples)][:]

self.labels_ = labels
self.centroids_ = centroids
return self
```

Figure 5. Centroids initialization from fit() method of KMeans.

3.3.2 Agglomerative Clustering

The **AgglomerativeClustering** class implements a bottom-up hierarchical clustering method. Its goal is to group samples by repeatedly merging the closest clusters until a chosen number of clusters is reached.

The algorithm starts with each sample as its own cluster and computes all pairwise distances. At each step, the two nearest clusters are merged using a selected linkage method (**single**, **complete**, or **average**). The linkage choice defines how distances between clusters are measured. More complex methods, such as Ward linkage, are not included to keep the implementation simple and clear.

3.3.3 SpectralClustering

The **SpectralClustering** class implements an advanced clustering method based on graphs. Its purpose is to detect complex, non-linear cluster structures that are hard to find with simple distance-based methods.

The algorithm builds an affinity matrix using an RBF kernel applied to pairwise distances. From this matrix, a normalized graph Laplacian is computed and its smallest eigenvectors are used to project the data into a lower-dimensional space. This new representation highlights the cluster structure.

```
def fit(self, X, y=None):
    # ---- validation ----
    if X is None:
        raise ValueError("X cannot be None")
    n = len(X)
    if n == 0:
        raise ValueError("X cannot be empty")
    if self.n_clusters <= 0:
        raise ValueError("n_clusters must be a positive integer")
    if self.n_clusters > n:
        raise ValueError("n_clusters cannot be greater than number of samples")
    if self.gamma <= 0:
        raise ValueError("gamma must be > 0")

    # ---- convert X to list of points ----
    X_list = [_as_point(X[i]) for i in range(n)]
    d = len(X_list[0])
    if d == 0:
        raise ValueError("Samples must have at least 1 feature")
    for i, p in enumerate(X_list):
        if len(p) != d:
            raise ValueError(f"Inconsistent feature dimensions at index {i}: expected {d}, got {len(p)}")

    # ---- build affinity matrix W ----
    dist_fn = _get_distance(self.metric)
    W = _rbf_affinity(X_list, dist_fn, gamma=self.gamma)

    # ---- compute normalized Laplacian L = I - D^{-1/2} W D^{-1/2} ----
    try:
        import numpy as np    # for eigen decomposition
    except ImportError:
        raise ImportError("SpectralClustering requires numpy (for eigen decomposition).")

    Wn = np.array(W, dtype=float)
    degrees = Wn.sum(axis=1)
    # avoid division by zero
    degrees[degrees == 0.0] = 1.0
    D_inv_sqrt = np.diag(1.0 / np.sqrt(degrees))

    I = np.eye(n)
    L = I - D_inv_sqrt @ Wn @ D_inv_sqrt # symmetric
```

Figure 6A. fit() method of SpectralClustering implementation.

The embedded data is then clustered using the existing **KMeans** implementation. This approach shows good modular design, as spectral clustering reuses k-means instead of redefining a new clustering step.

```
I = np.eye(n)
L = I - D_inv_sqrt @ Wn @ D_inv_sqrt # symmetric

# ---- eigenvectors for k smallest eigenvalues ----
eigvals, eigvecs = np.linalg.eigh(L)
U = eigvecs[:, : self.n_clusters] # n x k

# ---- row-normalize embedding ----
U_list = U.tolist()
U_norm = _row_normalize(U_list)

# ---- cluster rows with KMeans ----
km = KMeans(
    n_clusters=self.n_clusters,
    random_state=self.random_state,
    max_iter=self.max_iter,
    metric="euclidean", # kmeans in embedding space
)
km.fit(U_norm)

self.labels_ = km.labels_
return self
```

Figure 6B. fit() method of SpectralClustering implementation.

Once the fitting process ends for these clustering methods, final cluster labels are stored in the **labels_** attribute.

3.4 Datasets & Data Loading

*Dataset, BlobsDataset, IrisDataset, MNISTDataset, TGCADataset –
upsaclay_ml/datasets.py*

The datasets module provides a unified way to load, store, and access data across the library. Its main goal is to standardize how feature matrices and optional target labels are represented, so that models, transformers, and pipelines can operate on datasets without being tied to a specific file format or data source.

The **Dataset** class handles tabular data from CSV files or in-memory (**X**, **y**) arrays. When loading CSVs, it supports headers, different delimiters and encodings, and flexible target selection via **target_col**. Values are automatically cast to integers, floats, or kept as strings. Features are stored in **data** and labels in **target**. The class also supports **__len__**, **__getitem__** (with slicing), iteration yielding (**X**, **y**), and readable string representations.

BlobsDataset creates synthetic clustering data. **IrisDataset** downloads and encodes the Iris CSV dataset. **MNISTDataset** downloads MNIST, parses the files, and can normalize pixels. **TGCDataset** retrieves a small categorical table from the GDC API without labels.

The module keeps data handling separate from models and ensures a consistent (**X**, **y**) interface, making it easy to use transformers, models, and pipelines on local files, benchmarks, synthetic data, or API datasets.

3.5 Transformers

Transformer, Binarizer, StandardScaler, MinMaxScaler, OneHotEncoder – [upsaclay_ml/transformers.py](https://github.com/upsaclay/ml/transformers.py)

Transformers are used to preprocess data before it is passed to learning algorithms. In the library, transformers are designed to follow the same interface as estimators so that they can be chained seamlessly inside a pipeline. This design choice ensures that preprocessing steps and models can be composed in a uniform and intuitive way.

The **Transformer** class extends **Estimator** and adds **transform(X)** and **fit_transform(X)** methods. **fit()** learns any needed parameters, while **transform** applies the transformation. This consistent API makes it easy to use transformers in pipelines.

Binarizer converts numerical values to binary using a threshold. **StandardScaler** standardizes features to zero mean and unit variance, storing **mean_** and **std_**. **MinMaxScaler** rescales features to a fixed range, usually [0,1]. **OneHotEncoder** converts categorical values to one-hot vectors, optionally learning categories from the data.

All transformers follow the same conventions, storing learned parameters with **_** and supporting pipelines. This allows preprocessing steps to be combined flexibly with models for reusable workflows.

```
class MinMaxScaler(Transformer):
    """
    Scales features to a fixed range [minimum, maximum]
    """
    def __init__(self, minimum=0, maximum=1):
        self.minimum = minimum
        self.maximum = maximum
        self.min_ = None
        self.max_ = None

    def fit(self, X):
        """
        Computes min and max values for each feature
        """
        n_features = len(X[0])
        self.min_ = []
        self.max_ = []

        for j in range(n_features):
            column = [row[j] for row in X]
            self.min_.append(min(column))
            self.max_.append(max(column))

        return self

    def transform(self, X):
        """
        Apply Min-Max scaling using learned min and max
        """
        transformed = []
        for row in X:
            min_maxed_row = []
            for j in range(len(row)):
                x = row[j]
                min_maxed = (self.maximum - self.minimum) * (x - self.min_[j]) / (self.max_[j] - self.min_[j] + 1e-8) + self.minimum
                min_maxed_row.append(min_maxed) # It's kinda the same logic as StandardScaler
            transformed.append(min_maxed_row)

        return transformed
```

Figure 7. MinMaxScaler implementation.

3.6 Model Selection

[train_test_split - upsaclay_ml/model_selection.py](#)

Model selection utilities provide basic tools for evaluating how well a model generalizes to unseen data. In the library, this functionality is intentionally kept minimal and explicit, focusing on clarity rather than covering a wide range of advanced validation strategies.

The **train_test_split** function implements a simple random split of a dataset into training and testing subsets. Feature vectors and labels are first paired together to preserve their correspondence, then shuffled randomly before splitting. A fixed proportion of the data is reserved for testing, while the remaining samples are used for training. This approach helps reduce overfitting by ensuring that model evaluation is performed on data that was not seen during training.

```
def train_test_split(X, y):
    """
    Splits X and y into training and test sets
    X: Features
    y: Labels

    Returns X_train, X_test, y_train, y_test after splitting
    """
    pairs = []
    # Making pairs of label/feature
    for i in range(len(X)): # len(X) = len(y)
        pairs.append((X[i], y[i]))
    random.shuffle(pairs) # Gambling ahh shuffle

    test_size = 0.2 # You can change the value
    test_count = int(len(X) * test_size)
    # Getting the good index so we can split
    X_test = [pair[0] for pair in pairs][:test_count]
    y_test = [pair[1] for pair in pairs][:test_count]
    X_train = [pair[0] for pair in pairs][test_count:]
    y_train = [pair[1] for pair in pairs][test_count:]

    return X_train, X_test, y_train, y_test
```

Figure 8. train test split() method implementation.

This utility is designed to integrate smoothly with the rest of the library, returning data in a format directly compatible with estimators, pipelines, and evaluation metrics. While deliberately simple, it provides a clear and reproducible baseline for model evaluation and can easily be extended in future iterations to support more advanced strategies such as cross-validation or stratified sampling.

3.7 Metrics

[upsaclay_ml/metrics.py](#)

Evaluation metrics are used to quantify the performance of models implemented in the library. The metrics module provides a set of functions covering both supervised and unsupervised learning scenarios, allowing models to be evaluated consistently and independently from their internal implementations.

For clustering, external validation metrics require ground-truth labels. These include the **Rand Index** and **Adjusted Rand Index**, which measure agreement between true and predicted clusters, accounting for chance. Other metrics like **homogeneity**, **completeness**, and **V-measure** evaluate how well samples of the same class are grouped. Without ground-truth labels, the **silhouette score** measures cluster compactness and separation.

For supervised learning standard metrics are available. **Mean squared error** evaluates regression predictions, while **accuracy** measures classification performance. All metrics are implemented as standalone functions, independent of specific models.

```
def adjusted_rand_score(X, Y, predicted_labels):
    """
    Returns the Adjusted Rand Index (ARI) between true labels and predicted labels
    https://scikit-learn.org/stable/modules/generated/sklearn.metrics.adjusted_rand_score.html
    ARI corrects the Rand Index : taking into account the expected number of correct pairs
    if clusters were random and adjusts the score
    by subtracting the expected value and normalizing
    """

    # We compute the number of pairs in a group of size k
    def count_pairs(k):
        return k * (k - 1) // 2

    n = len(Y) # number of points in labels
    if n <= 1: # If we have 0 or 1 point, there are no pairs, score = 0
        return 0 # Exactly like in rand_score

    # Counters
    contingency = {} # counts of (true_label, predicted_label) pairs
    true_count = {} # counts of each true label
    pred_count = {} # counts of each predicted label

    for t, p in zip(Y, predicted_labels):
        contingency[(t, p)] = contingency.get((t, p), 0) + 1
        true_count[t] = true_count.get(t, 0) + 1
        pred_count[p] = pred_count.get(p, 0) + 1

    # Compute number of agreeing pairs
    # Agreeing pairs = the pairs in common between true and predicted clustering
    a = sum(count_pairs(v) for v in contingency.values())
    # same cluster & same true label
    b = sum(count_pairs(v) for v in true_count.values())
    # total pairs in same true class
    c = sum(count_pairs(v) for v in pred_count.values())
    # total pairs in same predicted cluster

    total_pairs = count_pairs(n) # total possible pairs
    expected = (b * c) / total_pairs # number of agreeing pairs we would expect by chance
    max_index = (b + c) / 2 # maximum possible number of agreeing pairs given the cluster sizes

    # Avoid division by zero if maximum = expected because we do max_index - expected
    if max_index == expected:
        return 0

    # Adjusted Rand Index = (observed - expected) / (max - expected)
    return (a - expected) / (max_index - expected)
```

Figure 9. adjusted_rand_score() method implementation.

3.8 Pipeline

Pipeline – upsacalay_ml/pipeline.py

The **Pipeline** class allows to chain multiple preprocessing steps with a final model into a single workflow. Its goal is to simplify applying transformations and training or predicting with a model in the correct order.

A pipeline is an ordered list of steps: all except the last are transformers, and the final step is an estimator. During training, each transformer is fitted and applied in sequence, and the final estimator is trained on the transformed data. During inference, only the transformations are applied before calling the final estimator's **predict** method.

The pipeline is callable like any estimator, making it easy to use. By combining preprocessing and modeling into one object, pipelines create modular, reusable workflows that work smoothly with transformers, models, and metrics.

```
class Pipeline:
    """
    Simple pipeline for chaining transformers and a final estimator
    Each step is a tuple: (name, transformer/estimator)
    """

    def __init__(self, steps):
        self.steps = steps
        self.transformers = steps[:-1] # all but last
        self.estimator = steps[-1][1] # the last step object

    def fit(self, X, y=None):
        """
        Fit all transformers and then the final estimator
        """
        Xt = X
        for name, step in self.transformers:
            step.fit(Xt)
            Xt = step.transform(Xt)

        # Fit the final estimator
        if y is not None:
            self.estimator.fit(Xt, y)
        else:
            self.estimator.fit(Xt)
        return self

    def predict(self, X):
        """
        Transform X through all transformers, then call estimator.predict
        """
        Xt = X
        for name, step in self.transformers:
            Xt = step.transform(Xt)
        return self.estimator.predict(Xt)

    # Optional: allow calling pipeline like a function
    def __call__(self, X):
        return self.predict(X)
```

Figure 10. Pipeline Class implementation.

IV. LIBRARY EVALUATION & PERFORMANCE

4.1 Basic tests

To ensure the library's stability, basic tests were implemented. Test files were created and automatically run by GitHub to verify that the code works as expected.

```
from upsaclay_ml.models.cluster import KMeans
from upsaclay_ml.datasets import Blobs
from upsaclay_ml.metrics import silhouette_score

def test_unsupervised():
    X = Blobs(n_samples=200, centers=3)
    kmeans = KMeans(n_clusters=3)
    labels = kmeans.fit_predict(X)
    print("Silhouette:", silhouette_score(X, labels))

if __name__ == "__main__":
    test_unsupervised()

from importlib.resources import files

from upsaclay_ml.pipeline import Pipeline
from upsaclay_ml.models.linear_model import LinearRegression
from upsaclay_ml.transformers import StandardScaler, OneHotEncoder
from upsaclay_ml.datasets import Dataset
from upsaclay_ml.metrics import mean_squared_error

def test_pipe():
    csv_path = files("upsaclay_ml").joinpath("test_dataset.csv")

    # Label is last column, header exists
    ds = Dataset(str(csv_path), has_header=True, target_col=-1)
    X, y = ds # X: [id, age, gender, height_cm, weight_kg]

    GENDER_COL = 2

    # 1) One-hot encode gender (needs 2D n x 1)
    X_gender = [[row[GENDER_COL]] for row in X]
    enc = OneHotEncoder()
    X_gender_oh = enc.fit_transform(X_gender)

    # 2) Numeric part: drop id (col 0) and drop gender (col 2) numeric columns in X are: age (1), height (3), weight (4)
    X_num = [[row[1], row[3], row[4]] for row in X]

    # 3) Combine numeric + one-hot
    X_final = [xn + xoh for xn, xoh in zip(X_num, X_gender_oh)]

    pipe = Pipeline([
        ("scaler", StandardScaler()),
        ("model", LinearRegression()),
    ])

    pipe.fit(X_final, y)
    y_pred = pipe.predict(X_final)

    print("MSE:", mean_squared_error(y, y_pred))

if __name__ == "__main__":
    test_pipe()
```

Figure 11B. Test files meant to be ran by GitHub.
Other files were created, and can be found on our repository.

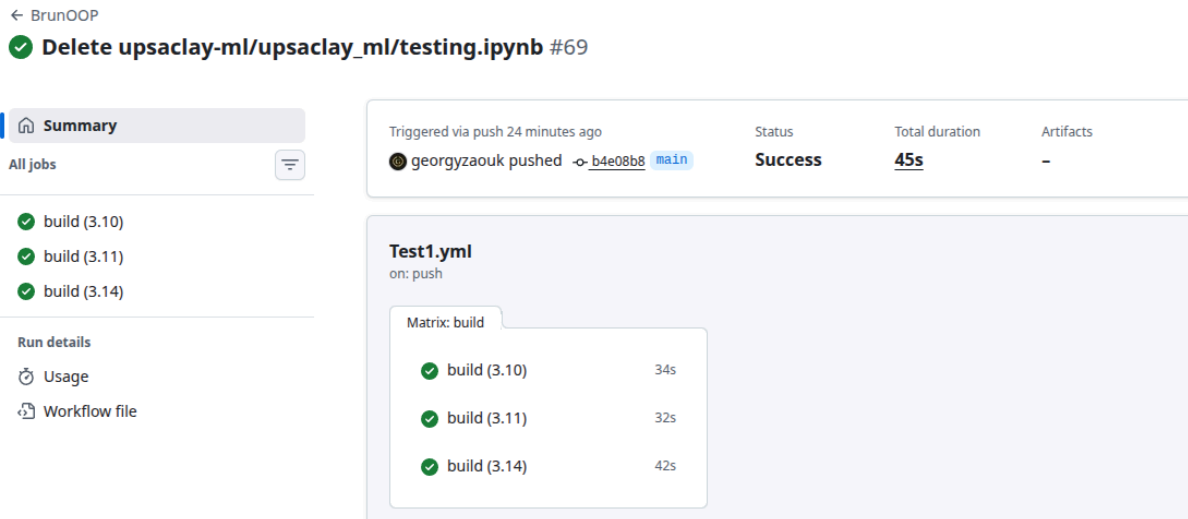


Figure 11A. GitHub actions tests results.
(All jobs were successfully executed)

Since GitHub confirms the library is stable for Python versions 3.10, 3.11, and 3.14, we can benchmark it and compare the results with the Sklearn module.

4.2 Performance & Comparison with Sklearn

The following results were obtained using **test_BrunOOP.py** (corresponding to our model) and **test_BrunOOP_SK.py** (respectively corresponding to the Sklearn equivalent of our model). BrunOOP is the name of our library.

4.2.1 Regression & Classification Comparison

This test compares the supervised learning pipelines. The dataset used for the LinearRegression test is a local custom CSV file (test_dataset.csv). It contains 10 rows and 5 columns, including features like age and gender ('M'/'F'). The size of this dataset has a huge impact on the results. Also for the classification test, with the DecisionTreeClassifier, IrisDataset was used.

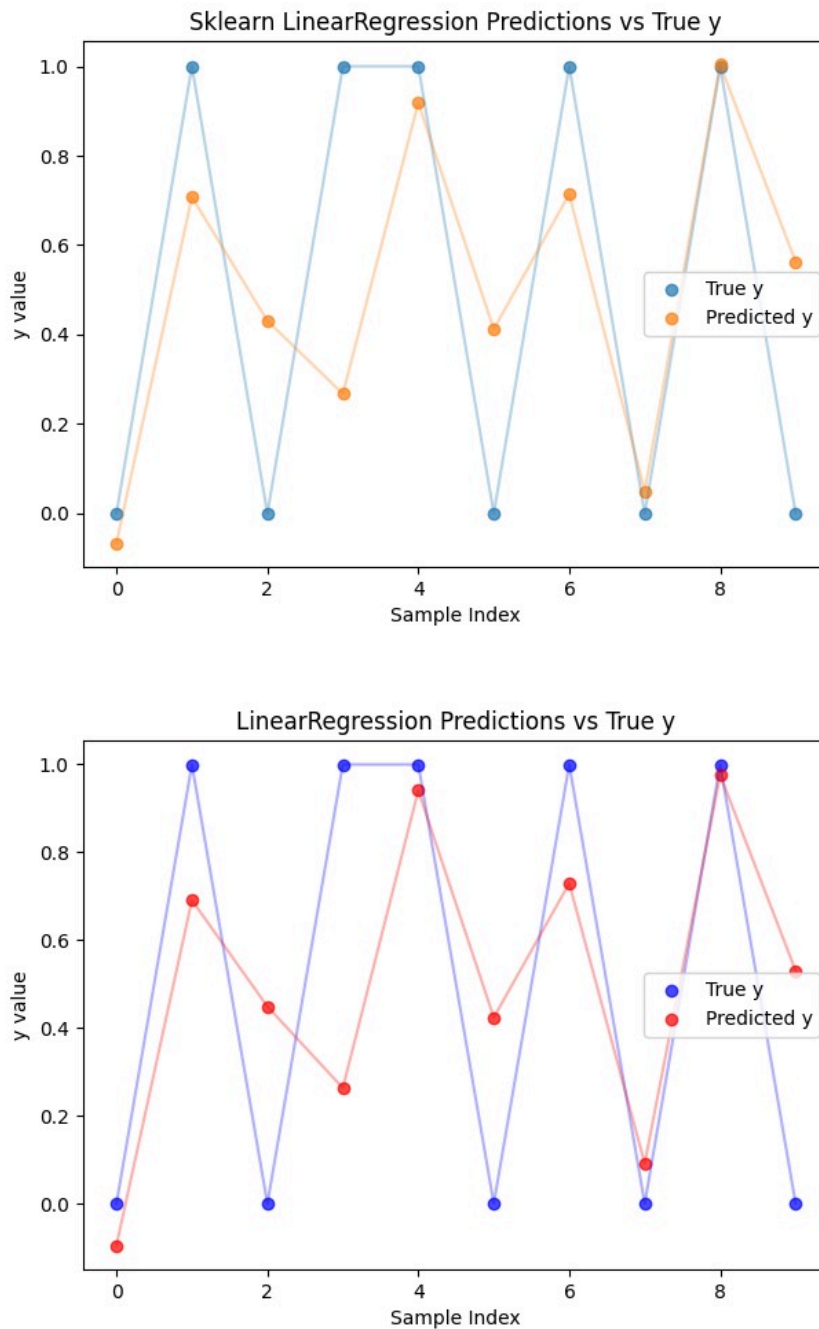


Figure 12. Graphs of LinearRegression predictions : Sklearn (Top) & BrunOOP (Bottom)

The model shows a strong correlation with the ground truth and captures most of the data's variation across ten samples. While it follows the overall trend well, the linear nature of the estimator leads to small errors at the extreme binary values. We should also highlight that both graphs are identical, meaning that LinearRegression was well implemented in BrunOOP.

Test Case	Metric	BrunOOP	Sklearn	Note
Test 1: Regression	MSE	0.1395	0.1389	Sklearn is slightly more accurate.
	Fit Time	0.0073s	0.0237s	BrunOOP was significantly faster.
Test2: Classification	Accuracy	0.9667	1.0000	Sklearn achieved perfect accuracy.
	Fit Time	0.0030s	0.0044s	Both are extremely fast.

Table 1. Regression & Classification Comparison.

For both tests, metrics show pretty much the same results. Indeed, as the metrics are slightly different, we can say that **LinearRegression** and **DecisionTreeClassifier** were well implemented in BrunOOP.

4.2.2 Unsupervised Learning Comparison (Clustering)

This test compares how well the clustering algorithms performed against the ground truth labels of the **Blobs** dataset. As Sklearn's **Blobs** and our implementation of Blobs are obviously different, it is quite difficult to deduce if BrunOOP did well. Also, the metrics are differently computed, and we believe that BrunOOP's metrics aren't correctly computed, besides the Silhouette score and Accuracy score.

Algorithm	Metric	BrunOOP	Sklearn	Change
KMeans	Silhouette	0.6212	0.4994	BrunOOP clusters are better defined.
	Adjusted Rand Index	0.9850	0.8427	BrunOOP matches ground truth better.
Agglomerative	Silhouette	0.7725	0.7049	BrunOOP is higher quality.
	V-Measure	1.0000	0.9192	BrunOOP achieved a perfect match.
Spectral	Silhouette	0.7462	0.5273	Significant drop in Sklearn.
	V-Measure	1.0000	0.7954	BrunOOP achieved a perfect match.

Table 2. Unsupervised Learning Comparison (Clustering).

While BrunOOP seems to perform well here, it is quite obvious that the metrics used for our model aren't correctly computed, especially V-Measure, as a V-Measure of 1 is nearly impossible to obtain with a dataset such as Blobs. However, as said earlier the Sklearn's datasets are differently implemented than our model's. This difference can explain the high values of Silhouette and ARI.

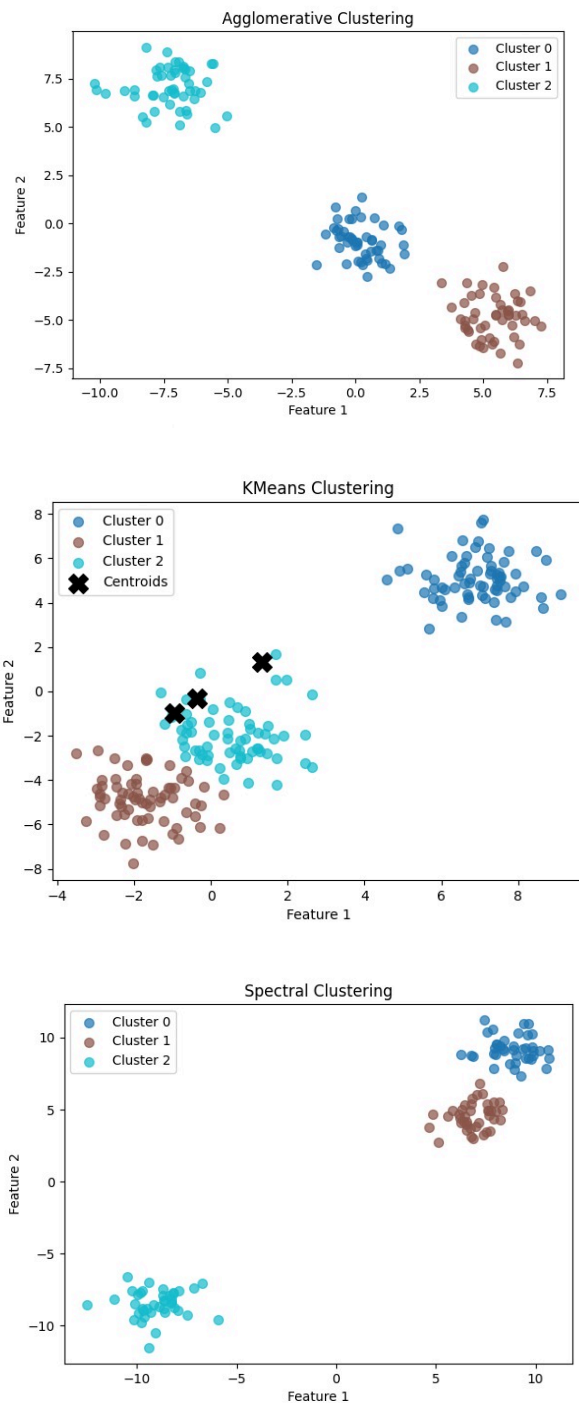


Figure 13A. Graphs of clusters for each BrunOOP algorithm :
Agglomerative (Top), KMeans (Middle) & Spectral (Bottom)

Since we were unsure whether the issue came from our **Blobs** dataset or from our metrics implementation, we evaluated the performance of the different clustering methods on the **Blobs** dataset, treating it like any other dataset. The figure above shows good clustering results.

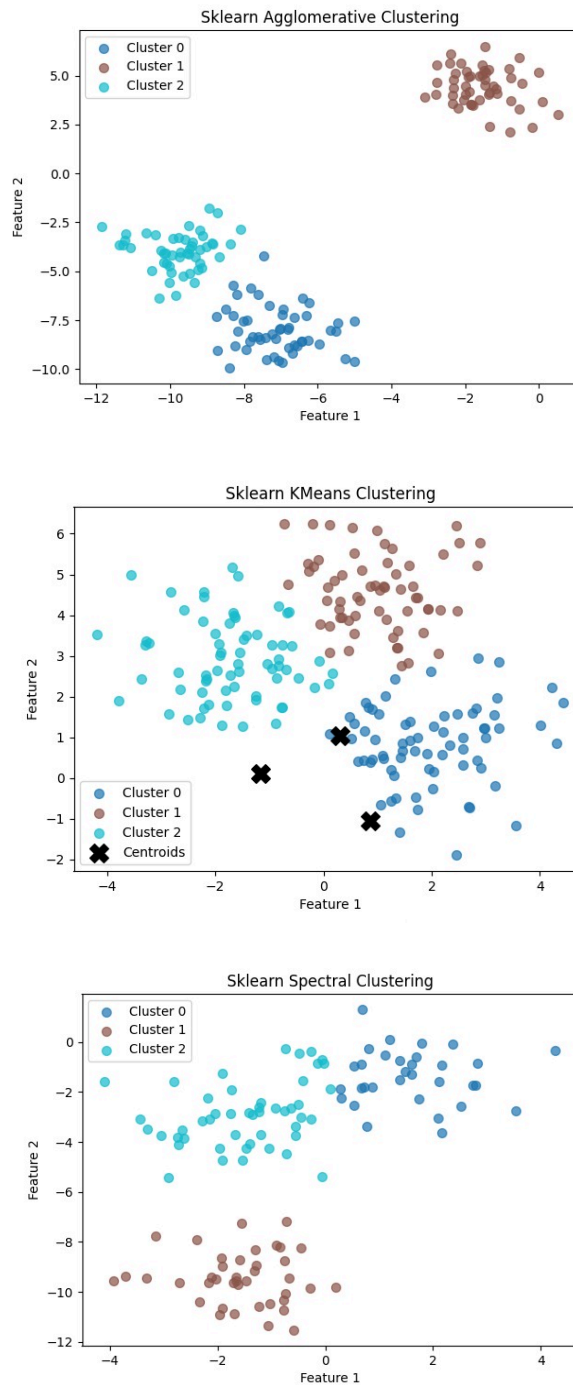


Figure 13B. Graphs of clusters for each Sklearn algorithm :
Agglomerative (Top), KMeans (Middle) & Spectral (Bottom)

The figure obtained with the Sklearn algorithms is very different from the one produced by BrunOOP. Although the clustering results are good, the clusters are distributed differently. This suggests that the **Blobs** dataset was implemented differently in BrunOOP. It is possible that the other datasets we implemented would show similar results.

4.2.3 Computational Performance

Here we wanted to check BrunOOP's speed, and compare it to Sklearn's speed for each test we made. Although our model is faster than Sklearn, the datasets and the accuracy of both libraries are different. Thus, this computational performance doesn't reveal anything about BrunOOP being better or worse than Sklearn, but simply shows how fast BrunOOP is.

Test Case	BrunOOP Time	Sklearn Time	Speed Winner
Test 1 (LR)	0.0073	0.0237	BrunOOP
Test 2 (DT)	0.0030	0.0044	BrunOOP
Test 3 (KMeans)	0.0029	0.0361	BrunOOP
Test 4 (Agglo)	0.1428	0.0246	Sklearn
Test 5 (Spectral)	0.1654	0.0211	Sklearn

Table 3. Execution time comparison.

V. CONCLUSION

This project focused on designing a small machine learning library, emphasizing object-oriented programming over performance. We built a modular architecture inspired by scikit-learn and implemented key components from scratch, including models, datasets, preprocessing tools, evaluation metrics, and a pipeline system for complete workflows.

One of the main strengths of the library is the clarity and consistency of its API. The unified Estimator structure, with a clear distinction between hyperparameters and learned attributes, allows models, transformers, and pipelines to work well together. This design is particularly useful for building pipelines, as preprocessing steps and learning algorithms can be easily combined. Overall, the architecture follows the object-oriented principles taught in the course and results in code that is readable, maintainable, and reusable.

However, the project also has several limitations. The implementations focus more on clarity and educational value than on computational efficiency. This is especially visible in clustering algorithms such as Agglomerative and SpectralClustering, whose performance decreases when the dataset size becomes large. In addition, the evaluation of unsupervised models revealed weaknesses in some metric implementations. For example, metrics like V-measure produced unrealistic results, which suggests possible errors in their computation or in the use of true labels. This reduces the reliability of some performance comparisons.

Another limitation concerns dataset management and scalability. Although the Dataset structure provides a unified way to handle CSV files, synthetic datasets, and external sources such as **MNIST** and **TCGA**, it does not include advanced features like memory-efficient loading or optimized batch processing. As a result, the library is mainly suitable for small to medium-sized datasets and for educational use, rather than for large-scale or production environments. Moreover, the datasets implemented in Sklearn are quite different from those in BrunOOP (our library), as shown above with the **Blobs** example.

If the project were extended, several improvements could be considered. These include correcting and validating the metric implementations, optimizing slow algorithms, adding cross-validation for model selection, and supporting model saving and loading. In conclusion, even though this library is not meant to compete with established machine learning frameworks, it successfully achieves its educational objectives. It demonstrates how object-oriented programming principles can be used to build a clean, modular, and extensible machine learning library, while also showing the trade-offs between code clarity, correctness, and performance.